123

# Intermediate Scaling Laws

Instructor: Hyunwoo J. Kim

# Announcements

123

- 6/2~6/4 Recorded Lectures on Advanced Topics
- 6/9 Scaling Laws Presentation
    - Top-Ranked Students' Presentations

# Today's agenda

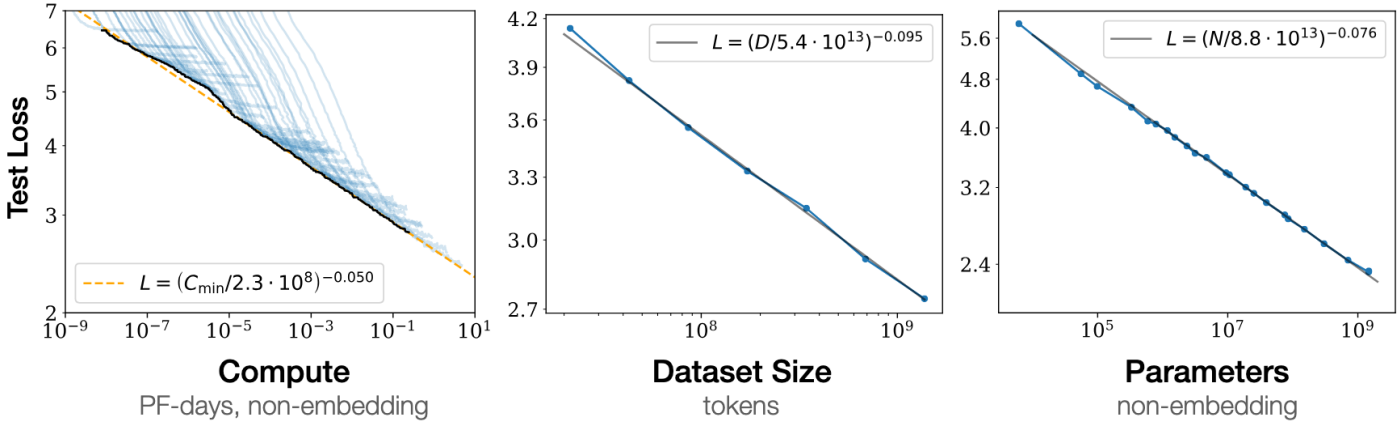
123

1. Scaling Law – Chinchilla
2. Data Repetition in Scaling Law
3. Hyperparameters in Scaling Law

# 1. Scaling Law - Chinchilla

# Foundations: Kaplan's Power-Law (2020)

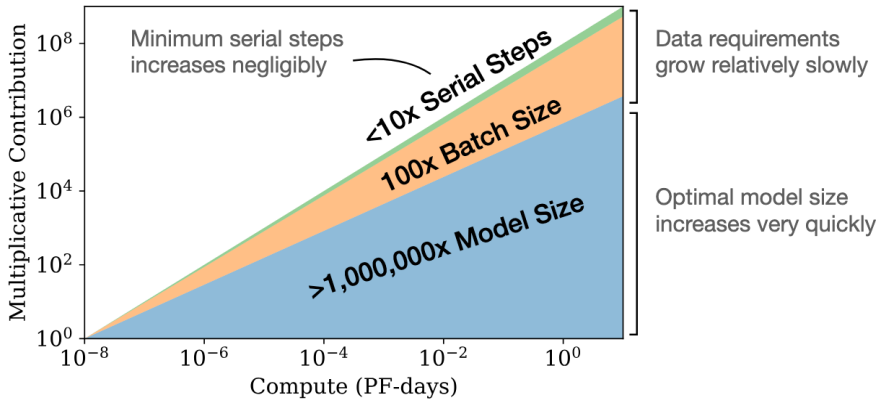
- A Scaling Law describes **how model loss or error changes** as the amount of **training resources increase**
- Scale (resources): **Compute (C), dataset size (D),** and **model parameters (N)**
  - power-law relationship




[1] Jared, Kaplan, et al. "Scaling Laws for Neural Language Models" arXiv, 2020.

# Kaplan's Verdict: Prioritizing Model

- **Parameter Dominance**: Performance improves most efficiently by scaling model size (*N*) rather than data (*D*)
- **Scaling Coefficients**
  - Model Size dominates: $N_{opt} \propto C^{0.73}$
  - Data grows slowly: $D_{opt} \propto C^{0.27}$

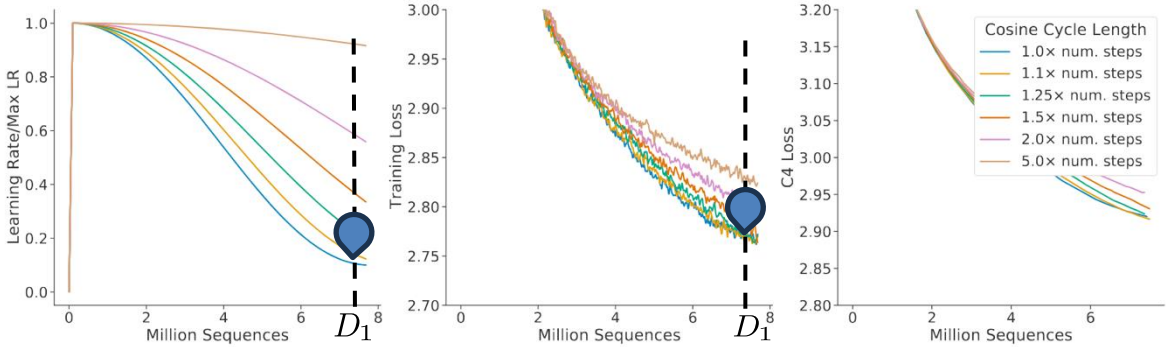



[1] Jared, Kaplan, et al. "Scaling Laws for Neural Language Models" arXiv, 2020.

# Revisiting Kaplan: The Fixed LR Schedule Issue

- **Fixed LR Schedule:** Kaplan used a **fixed cosine learning rate (LR) schedule** regardless of the actual number of training tokens (*D*)
  - Not Decaying LR  Model oscillates around the minimum  **Higher Loss**

- **Misleading Conclusion: scaling N is significantly more critical than *D***

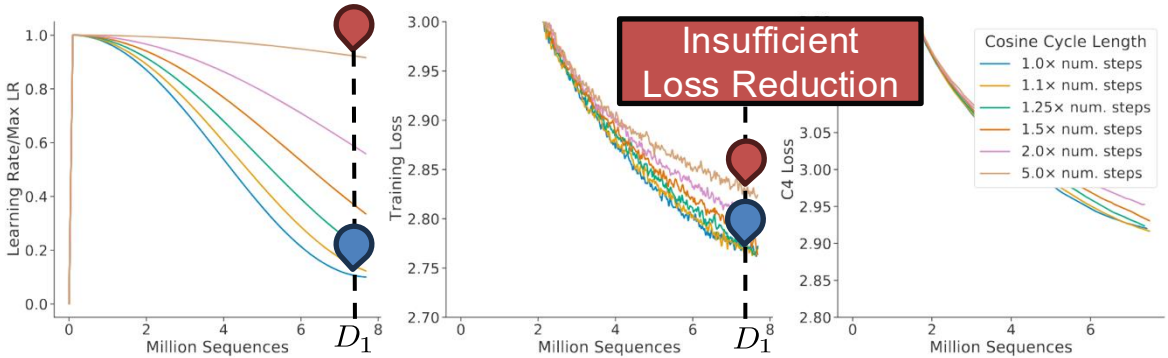



[1] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# Revisiting Kaplan: The Fixed LR Schedule Issue

- **Fixed LR Schedule:** Kaplan used a fixed cosine learning rate (LR) schedule regardless of the actual number of training tokens (*D*)
  - Not Decaying LR → Model oscillates around the minimum → Higher Loss

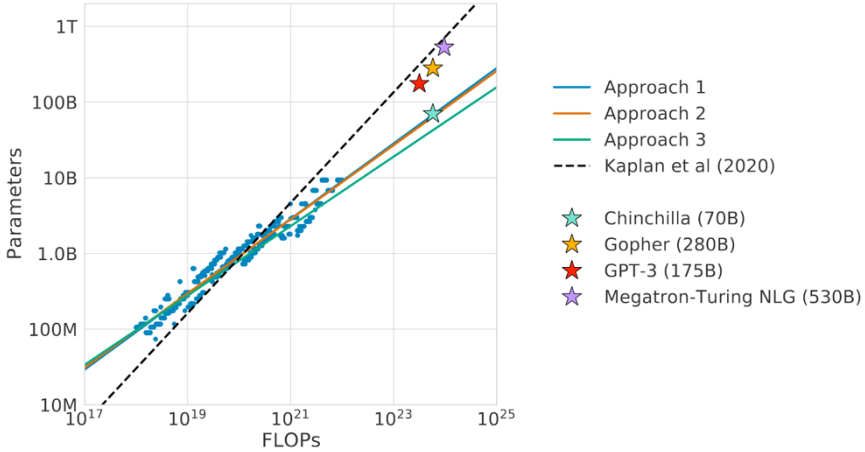
- **Misleading Conclusion:** scaling N is significantly more critical than *D*




[1] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# From Kaplan to Chinchilla

- **Key Message:** Current large models are **oversized** and **undertrained**
- **Revisiting Optimization:** Investigated the optimal trade-off between model size (N) and training tokens (D) for a fixed compute budget




| Model | Size (# Parameters) | Training Tokens |
|---|---|---|
| LaMDA (Thoppilan et al., 2022) | 137 Billion | 168 Billion |
| GPT-3 (Brown et al., 2020) | 175 Billion | 300 Billion |
| Jurassic (Lieber et al., 2021) | 178 Billion | 300 Billion |
| *Gopher* (Rae et al., 2021) | 280 Billion | 300 Billion |
| MT-NLG 530B (Smith et al., 2022) | 530 Billion | 270 Billion |
| *Chinchilla* | 70 Billion | 1.4 Trillion |

[1] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# Overview of Chinchilla's Methodology

123

Chinchilla's Methodologies

- Approach 1. Fix model sizes N and vary number of training tokens D
- Approach 2: IsoFLOP profiles (Vary model size for fixed compute budget)
- Approach 3: Fitting a parametric loss function

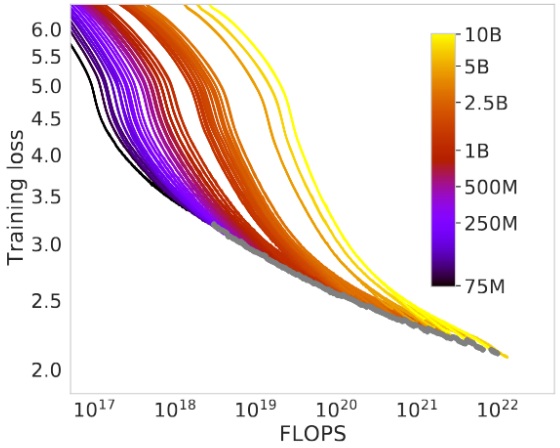
- The cosine LR was specifically matched to the number of training tokens
- Dataset: *MassiveText*, no more than one epoch

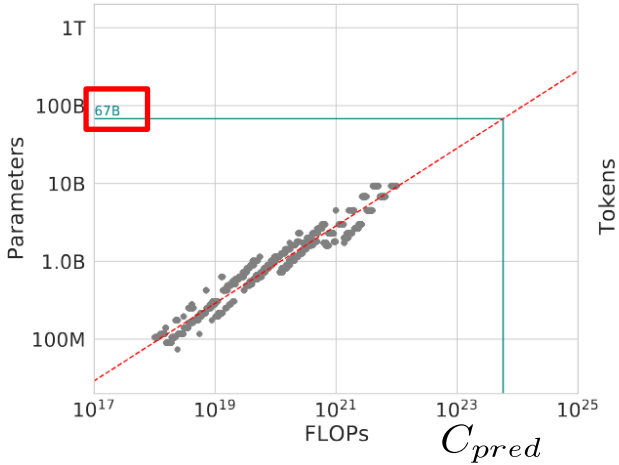
# Approach 1. Fix model sizes and vary number of training tokens

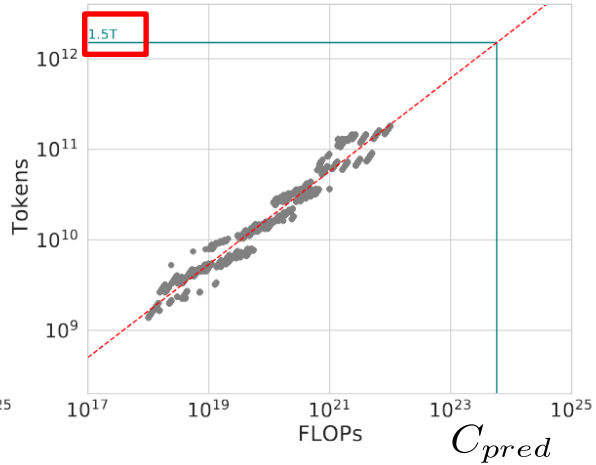
123

Analyze how **loss** scales by **fixing** *N* and **varying** *D*

- Train a fixed family of models ranging in [70M, 10B]
- Construct **an envelope of minimum loss** achieved for any given compute budget (FLOPs)
- Fit **power laws** to estimate optimal *N* and *D* for a given compute budget *C*



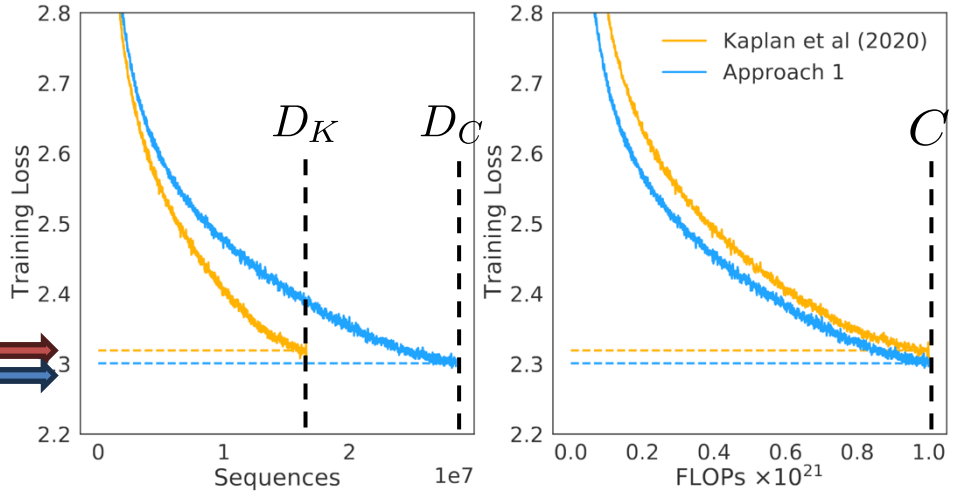




[1] Jordan, Hoffmann, et al. "Training Compute-Optimal Large Language Models" NeurIPS, 2022

# Approach 1. Fix model sizes and vary number of training tokens

123

Lower training loss




- **Result**: $N_{opt} \propto C^{0.50}, D_{opt} \propto C^{0.50}$
- **Validation of Approach 1 vs. Kaplan**
  - Budget: Fixed at $C = 10^{21}$ FLOPs
  - Kaplan's Prediction: $N_K = 4.74$B ( low $D_K$)
  - Chinchilla's Prediction: $N_C = 2.80$B ( high $D_C$)

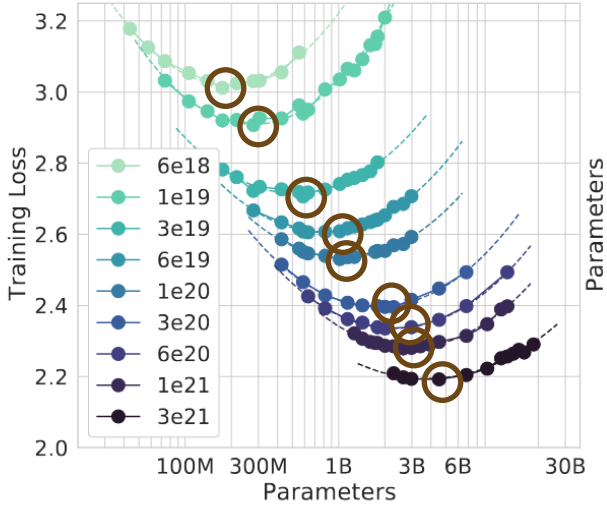
[1] Jordan, Hoffmann, et al. "Training Compute-Optimal Large Language Models" NeurIPS, 2022

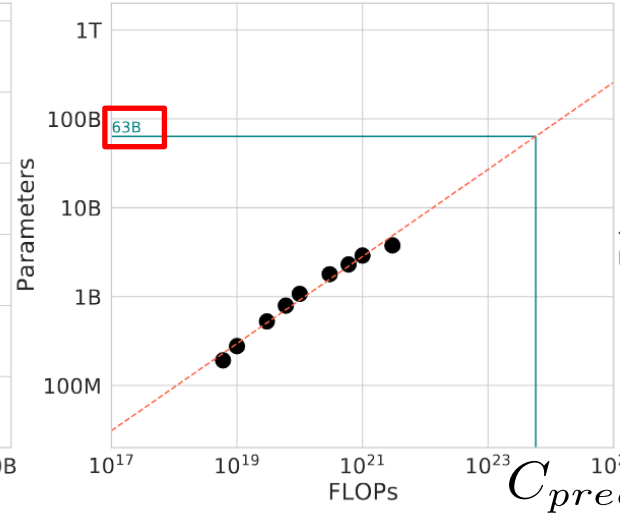
# Approach 2. IsoFLOP profiles

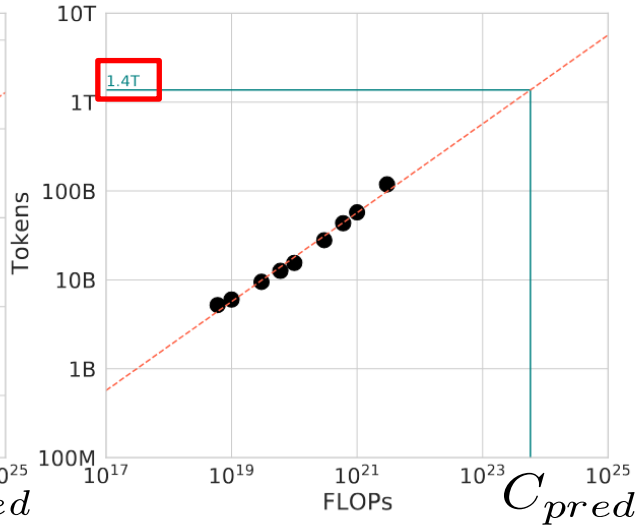


Determine the optimal model size for a **fixed compute budget**

- 9 distinct FLOP budgets in $[6\times10^{18}, 3\times10^{21}]$
- **Vary model size** while adjusting training tokens to **keep total FLOPs constant**
- **Identify the model size that minimizes validation loss** for each FLOP budget (the "valley" of the parabola) $N_{opt} \propto C^{0.49}, D_{opt} \propto C^{0.51}$



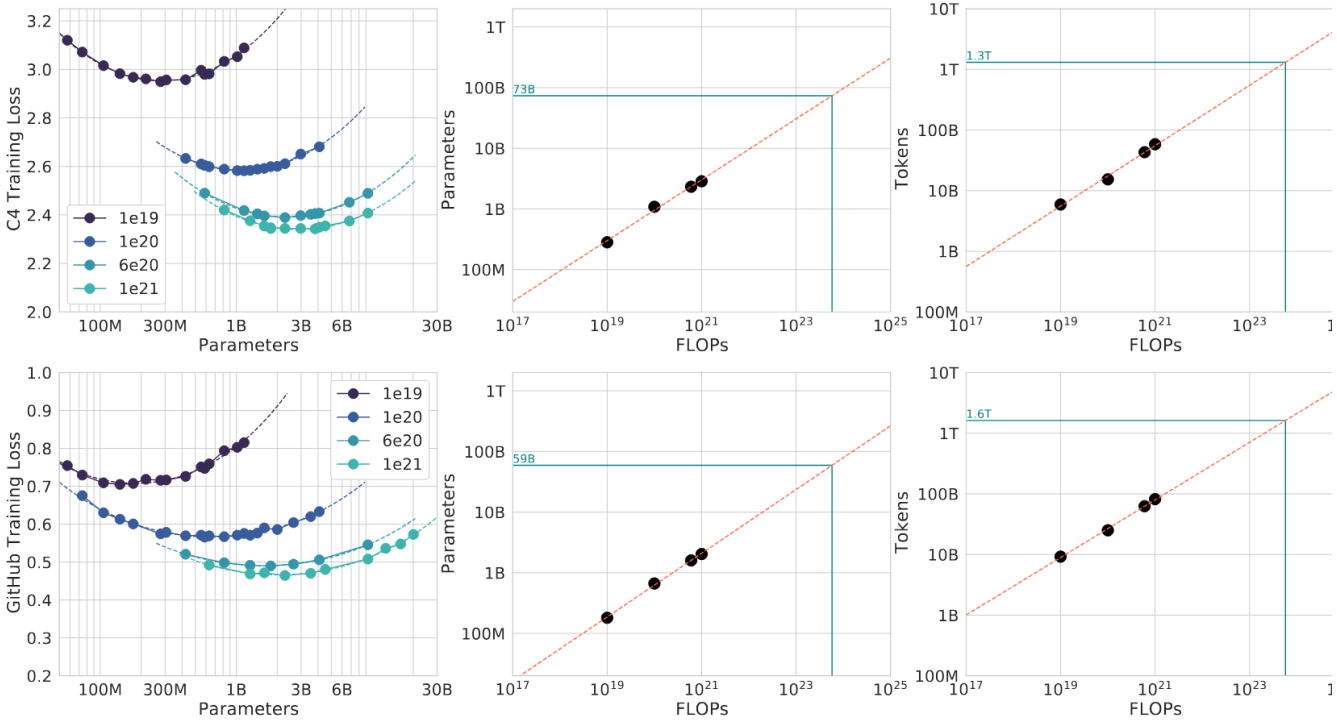




[1] Jordan, Hoffmann, et al. "Training Compute-Optimal Large Language Models" NeurIPS, 2022

# Approach 2. IsoFLOP profiles - Result

123




## Consistency of Approach 2

- IsoFLOP analysis on two different datasets (C4, Github code)
- Similar Scaling behavior and the coefficient

| Approach | Coef. $a$ where $N_{opt} \propto C^a$ | Coef. $b$ where $D_{opt} \propto C^b$ |
|---|---|---|
| C4 | 0.50 | 0.50 |
| GitHub | 0.53 | 0.47 |

[1] [2] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# Approach 3. Fitting a parametric function

123

**Model the loss as a parametric function** of model size (*N*) and tokens (*D*)

- **Propose a functional form:**
  - *E*: Irreducible loss (entropy of natural text)
  - First term: Loss due to finite model size
  - Second term: Loss due to finite data

$$\hat{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

# Approach 3. Fitting a parametric function

123

**Model the loss as a parametric function** of model size (*N*) and tokens (*D*)

- **Fit parameters** to minimize Huber loss across all runs (*A, B, E, [], []*)

$$\min_{A,B,E,\alpha,\beta} \sum_{\text{Runs } i} \text{Huber}_\delta \left( \log \hat{L}(N_i, D_i) - \log L_i \right)$$

- Derive the **efficient frontier** analytically by minimizing *L* under the constraint

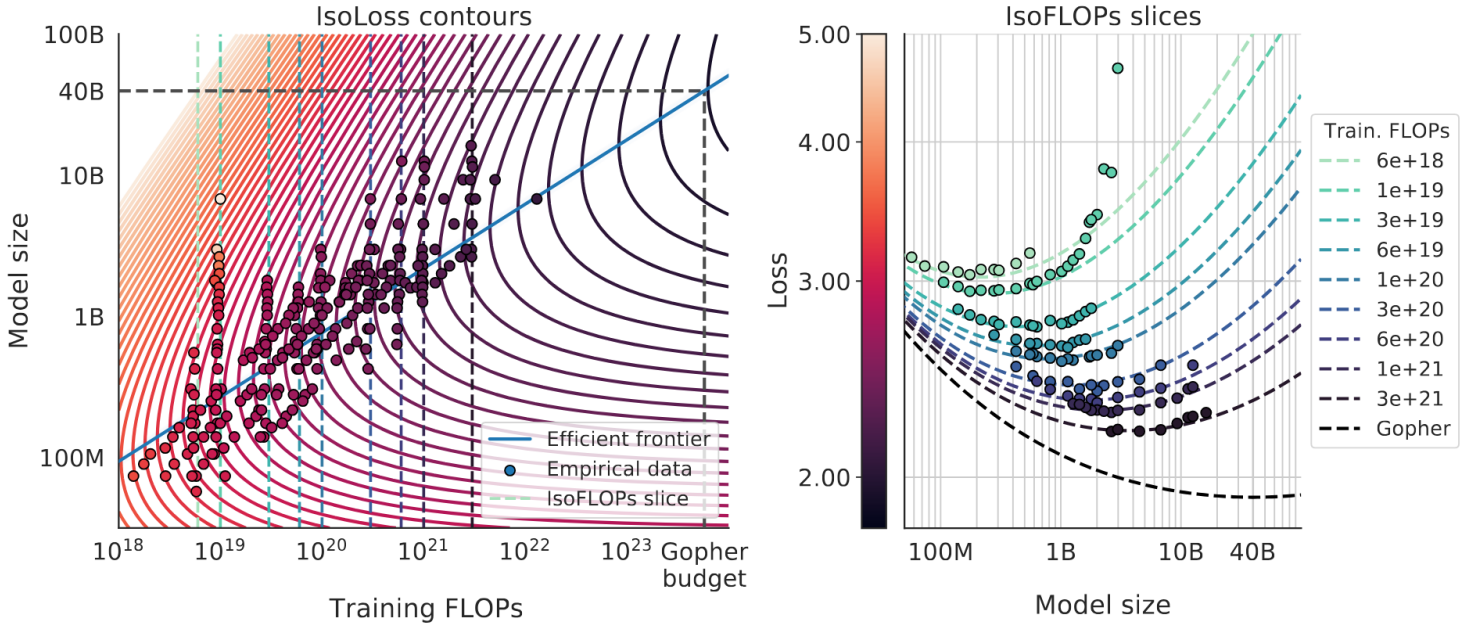
$$\hat{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

# Approach 3. Fitting a parametric function

123

## Parametric Fits

- IsoFLOP slices (dashed lines) show a clear minimum loss point for each budget
- The blue line: Efficient frontier




[1] Jordan, Hoffmann, et al. "Training Compute-Optimal Large Language Models" NeurIPS, 2022

# Approach 3. Fitting a parametric function - Derivation

123

## Derivation of Chinchilla Scaling Laws

### 1. Problem Setup

The goal is to minimize the loss function $\hat{L}(N, D)$ subject to a fixed computational budget $C$:

- **Objective Function:**

$$\hat{L}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

- **Constraint:**

$$C = 6ND \implies g(N, D) = 6ND - C = 0$$

### 2. Lagrange Multiplier Method

Define the Lagrangian function $\mathcal{L}$ by introducing the multiplier $\lambda$:

$$\mathcal{L}(N, D, \lambda) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} + \lambda(6ND - C)$$

### 3. First-Order Conditions

Set the partial derivatives with respect to $N$ and $D$ to zero:

$$\frac{\partial \mathcal{L}}{\partial N} = -\frac{\alpha A}{N^{\alpha+1}} + 6\lambda D = 0 \implies 6\lambda = \frac{\alpha A}{D N^{\alpha+1}} \quad (1)$$

$$\frac{\partial \mathcal{L}}{\partial D} = -\frac{\beta B}{D^{\beta+1}} + 6\lambda N = 0 \implies 6\lambda = \frac{\beta B}{N D^{\beta+1}} \quad (2)$$

### 4. Final Optimal Solution

Equating (1) and (2) yields the optimal ratio:

$$\frac{\alpha A}{N^{\alpha}} = \frac{\beta B}{D^{\beta}}$$

By substituting $D = \frac{C}{6N}$ into the above, we derive:

$$N_{opt}(C) = G\left(\frac{C}{6}\right)^{a}, \quad D_{opt}(C) = G^{-1}\left(\frac{C}{6}\right)^{b}$$

**Where constants are defined as:**

$$G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}, \quad a = \frac{\beta}{\alpha+\beta}, \quad b = \frac{\alpha}{\alpha+\beta}$$

# Approach 3. Fitting a parametric function - Result

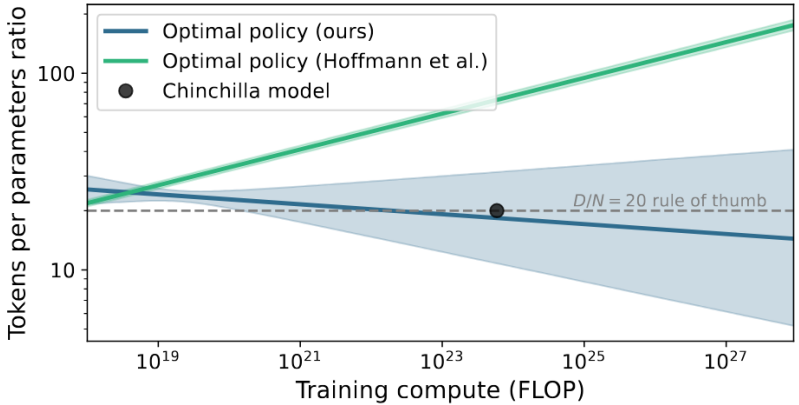
123

- **Result**: $N_{opt} \propto C^{0.46}, D_{opt} \propto C^{0.54}$
- Fitting a parametric loss function with just five coefficients (*A, B, E, □, □*) enables **the determination of the compute-optimal model size (*N*) and training tokens (*D*)**
- Approximately $D \approx 20N$ for Chinchilla model

# Approach 3. Fitting a parametric function - Result

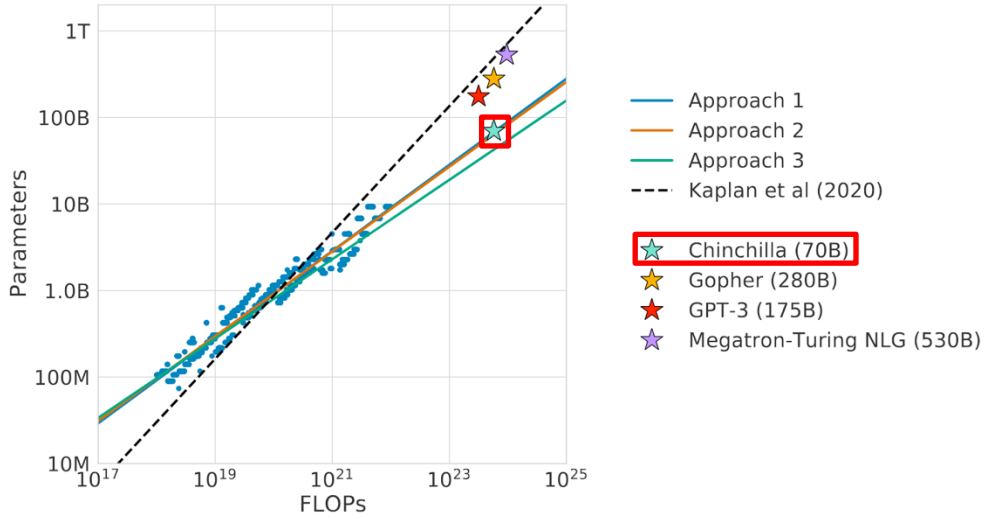
123

- **Result**: $N_{opt} \propto C^{0.46}, D_{opt} \propto C^{0.54}$
- **Fun addendum:** Besiroglu et al., 2024
  - **Curve Fitting Optimizer stopped early** due to incorrect loss scaling
  - **Corrected coefficients** (*a ≈ 0.51,b ≈ 0.49*) align with Approaches 1 & 2
  - **Reaffirms the Chinchilla's 1:1 scaling law**




[1] Tamay, Besiroglu, et al. "Chinchilla Scaling: A replication attempt" arXiv, 2024.

# Final Verdict: Chinchilla vs. Kaplan Coefficients




| Approach | Coeff. $a$ where $N_{opt} \propto C^a$ | Coeff. $b$ where $D_{opt} \propto C^b$ |
|---|---|---|
| 1. Minimum over training curves | 0.50 (0.488, 0.502) | 0.50 (0.501, 0.512) |
| 2. IsoFLOP profiles | 0.49 (0.462, 0.534) | 0.51 (0.483, 0.529) |
| 3. Parametric modelling of the loss | 0.46 (0.454, 0.455) | 0.54 (0.542, 0.543) |
| Kaplan et al. (2020) | 0.73 | 0.27 |

[1] [2] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# Evidence: Current LLMs are Oversized & Undertrained



- Current LLMs such as Gopher (280B) and GPT-3 (175B) are significantly **oversized** and **undertrained**
- To achieve optimal performance for a given compute budget, **model size (*N*)** and the number of **training tokens (*D*)** should be **scaled in equal proportions**

| Model | Size (# Parameters) | Training Tokens |
|---|---|---|
| LaMDA (Thoppilan et al., 2022) | 137 Billion | 168 Billion |
| GPT-3 (Brown et al., 2020) | 175 Billion | 300 Billion |
| Jurassic (Lieber et al., 2021) | 178 Billion | 300 Billion |
| *Gopher* (Rae et al., 2021) | 280 Billion | 300 Billion |
| MT-NLG 530B (Smith et al., 2022) | 530 Billion | 270 Billion |
| *Chinchilla* | 70 Billion | 1.4 Trillion |

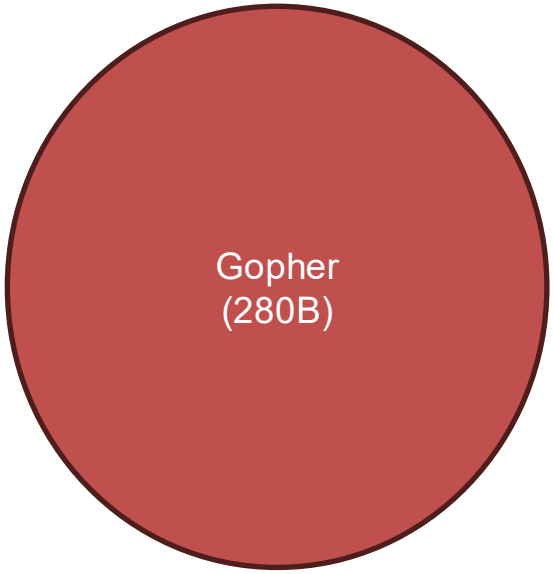
| Parameters | FLOPs | FLOPs (in *Gopher* unit) | Tokens |
|---|---|---|---|
| 400 Million | 1.92e+19 | 1/29,968 | 8.0 Billion |
| 1 Billion | 1.21e+20 | 1/4,761 | 20.2 Billion |
| 10 Billion | 1.23e+22 | 1/46 | 205.1 Billion |
| 67 Billion | 5.76e+23 | 1 | 1.5 Trillion |
| 175 Billion | 3.85e+24 | 6.7 | 3.7 Trillion |
| 280 Billion | 9.90e+24 | 17.2 | 5.9 Trillion |
| 520 Billion | 3.43e+25 | 59.5 | 11.0 Trillion |
| 1 Trillion | 1.27e+26 | 221.3 | 21.2 Trillion |
| 10 Trillion | 1.30e+28 | 22515.9 | 216.2 Trillion |

[1] [2] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# Downstream Bonus: Smaller Models, Faster Inference




(+) inference latency ↓
(+) memory footprint ↓




Facilitates **easier deployment on hardware with limited resources!**

- **Test-time Scaling Law** is also important (Lecture 3)

# Summary: Chinchilla



- **Paradigm Shift in Scaling:** Disproves Kaplan  **training data (D) is equally critical.**
  - Varying training tokens for fixed model sizes
  - IsoFLOP profiles (fixed compute budgets)
  - Fitting a parametric loss function

- **Resource Misallocation:** Identifies that current LLMs (e.g., GPT-3, Gopher) are significantly **"oversized and undertrained"**.

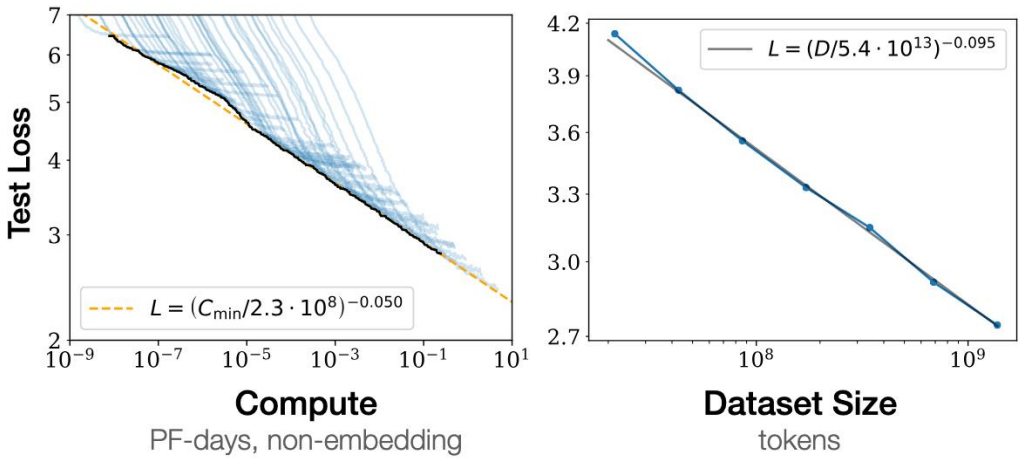
- **Optimal Strategy:** Scaling (Model Size):(Train token) = **1:1**

# 2. Data Constraints in Scaling Law

# The Ideal vs. Reality: Scaling with Finite Data

In Ideal,

The Single-Epoch Assumption: training data is unique and seen only once




[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# The Ideal vs. Reality: Scaling with Finite Data



In Ideal,

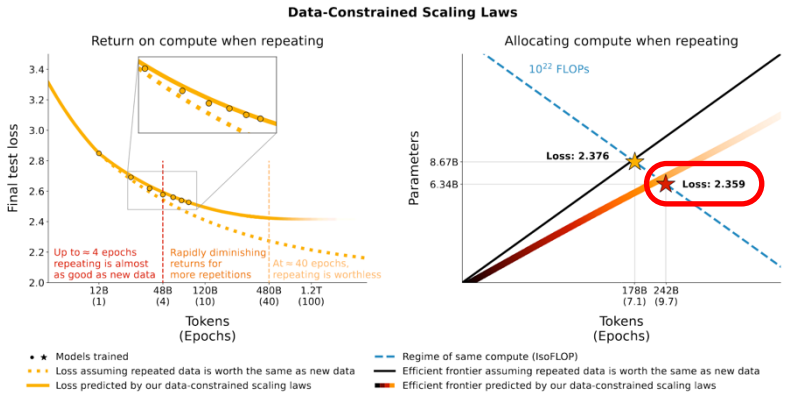
**The Single-Epoch Assumption:** training data is unique and seen only once

But in reality..

- If high-quality data is finite, can we **simply repeat** the dataset for multiple epochs without diminishing returns?
- How does the scaling law change when we are **forced to mix** high-quality data with lower-quality or synthetic data due to scarcity?

# Scaling Law of Data Repetition

- Repeating **maintains value for ~4 epochs**, but returns diminish to zero by ~40 epochs
- Shift **optimal allocation to smaller models trained for more epochs**
- **New scaling laws generalize Chinchilla** by modeling the value decay of repeated tokens




[1] Niklas, Muennighoff, et al. “Scaling Data-Constrained Language Models” NeurIPS, 2023.

# Modeling Effective Data in Repetition

123

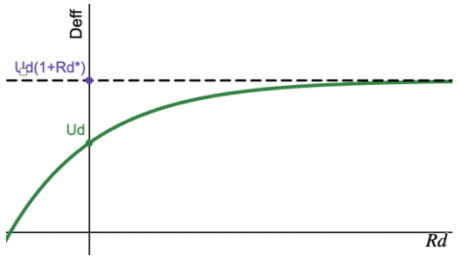
1. **Fit standard parameters** $A, B, E, \square, \square$ on single-epoch runs assuming $R_D = 0$ (by Chinchilla)
2. **Fix those terms, then fit decay constants** $R_D^*$, $R_N^*$ using multi-epoch training data

$$D' = U_D + U_D R_D^* \left(1 - e^{-\frac{R_D}{R_D^*}}\right)$$

- $D'$: Effective data, $U_D$: Unique tokens, $R_D^*$: Fitting Constant, $R_D$: Epoch

3. **Minimize Loss** $L(N', D')$ **under budget** $C \approx 6ND$, $U_D$ to find optimal $N$, $R_D$

□ Increasing repetition $R_D$ faster than parameter $N$ is better

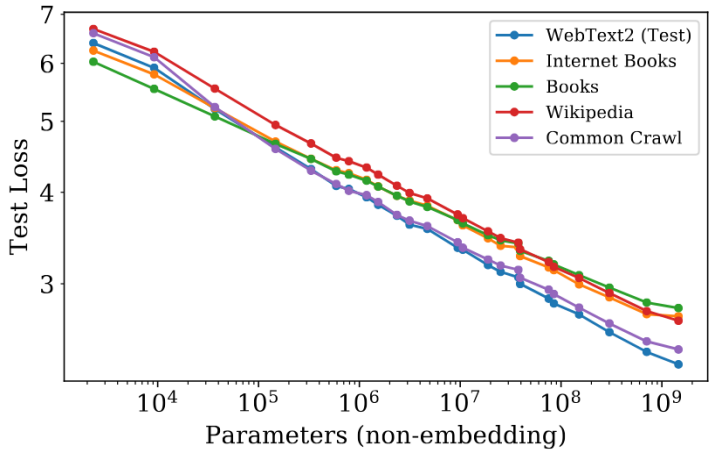



[1] Desmos. “A Parametric Study of D’ according to the change of Rd” Desmos Graphing Calculator, 2026. https://www.desmos.com/calculator

# Scaling Law of Data Composition



Datasets affect the offset of scaling law, **not the slope** [Kaplan, Chinchilla]




| Approach | Coef. $a$ ($N_{opt} \propto C^a$) | Coef. $b$ ($D_{opt} \propto C^b$) |
|---|---|---|
| MassiveText | **0.49** | **0.51** |
| C4 | 0.50 | 0.50 |
| GitHub | 0.53 | 0.47 |

[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.
[2] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022.

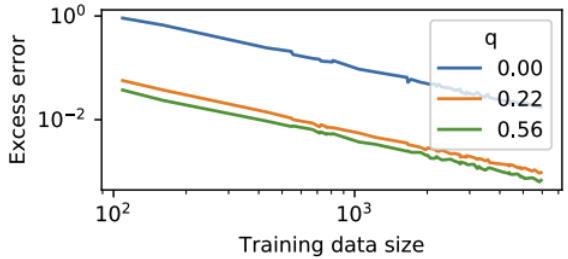
# Scaling Law of Data Composition

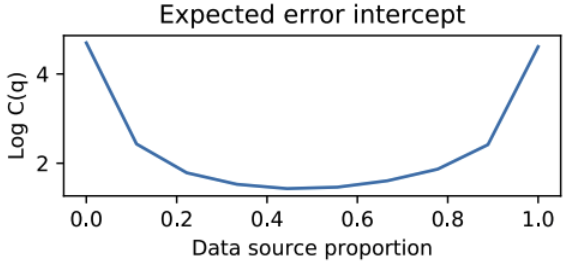
123

**Scaling slope also holds for mixtures; only alters the offset.** (Hashimoto, 2021)

- size-dependent power law
- mixture-dependent offset constant $C(q)$.

- **Small scales transfer effectively to larger one**
  - Find mixture-dependent term using few pilot runs

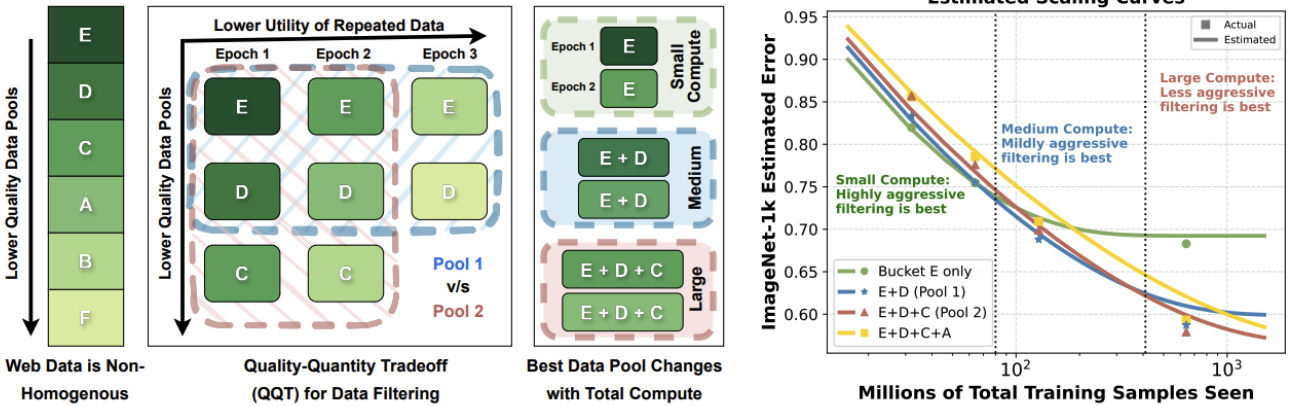



[1] Tatsunori, Hashimoto "Model Performance Scaling with Multiple Data Sources" ICML, 2021.

# Scaling Law of Data Repetition & Composition

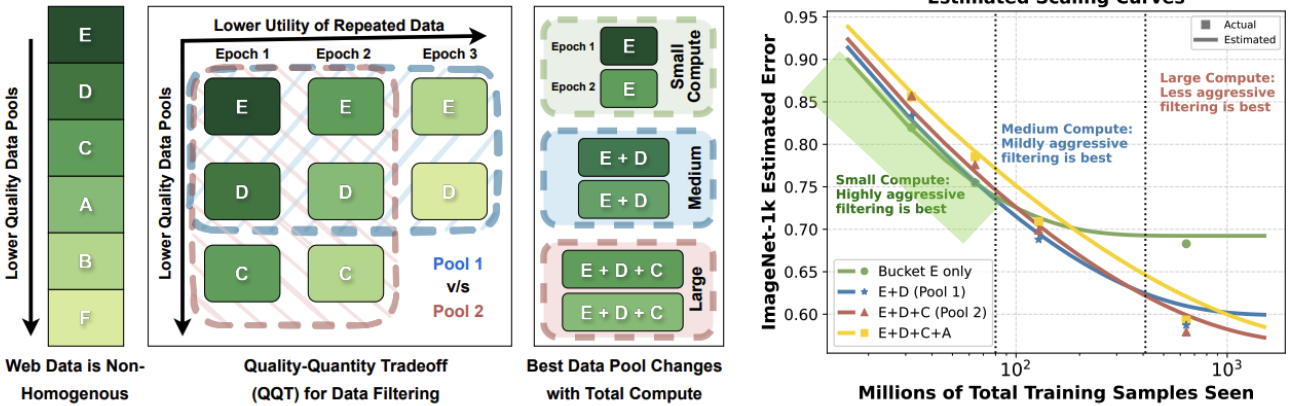
- **Optimal data curation:** depends on the compute budget, **cannot remain static**
- **Quality-quantity tradeoff**
  - Small budgets: Aggressive filtering suits
  - Large scales: Require adding lower-quality data
- New scaling laws (Goyal, 2024) **estimate mixture performance** w/o expensive training runs




[1] Sachin, Goyal, et al. “Scaling Laws for Data Filtering—Data Curation cannot be Compute Agnostic” CVPR (IEEE/CVF), 2024.

# Scaling Law of Data Repetition & Composition

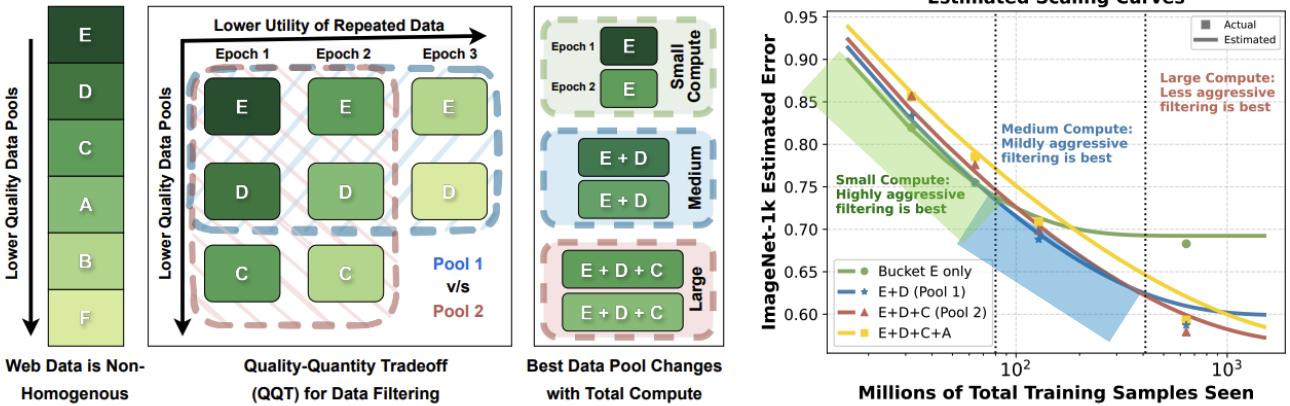
- **Optimal data curation:** depends on the compute budget, **cannot remain static**
- **Quality-quantity tradeoff**
  - Small budgets: Aggressive filtering suits
  - Large scales: Require adding lower-quality data
- New scaling laws (Goyal, 2024) **estimate mixture performance** w/o expensive training runs




[1] Sachin, Goyal, et al. "Scaling Laws for Data Filtering—Data Curation cannot be Compute Agnostic" CVPR (IEEE/CVF), 2024.

# Scaling Law of Data Repetition & Composition

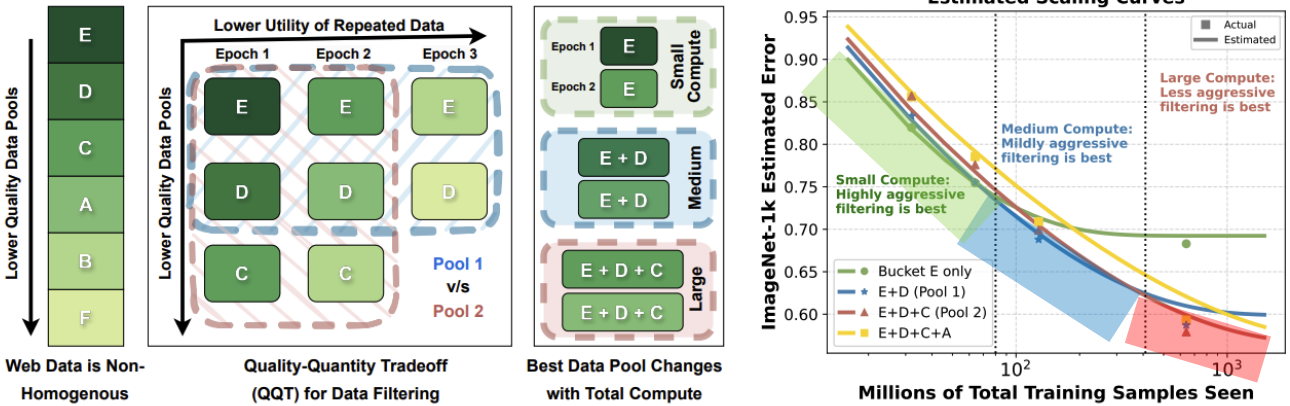
- **Optimal data curation:** depends on the compute budget, **cannot remain static**
- **Quality-quantity tradeoff**
  - Small budgets: Aggressive filtering suits
  - Large scales: Require adding lower-quality data
- New scaling laws (Goyal, 2024) **estimate mixture performance** w/o expensive training runs




[1] Sachin, Goyal, et al. “Scaling Laws for Data Filtering—Data Curation cannot be Compute Agnostic” CVPR (IEEE/CVF), 2024.

# Scaling Law of Data Repetition & Composition

- **Optimal data curation:** depends on the compute budget, **cannot remain static**
- **Quality-quantity tradeoff**
  - Small budgets: Aggressive filtering suits
  - Large scales: Require adding lower-quality data
- New scaling laws (Goyal, 2024) **estimate mixture performance** w/o expensive training runs




[1] Sachin, Goyal, et al. “Scaling Laws for Data Filtering—Data Curation cannot be Compute Agnostic” CVPR (IEEE/CVF), 2024.

# Summary: Scaling Law for Data Constraints

123

- Real-world training faces constraints of **data scarcity** and **heterogeneous quality**.
- Repeated training requires **modeling effective data due to diminishing returns**.
- **Data quality shifts the scaling curve's intercept.**
- Optimal compute allocation varies with data quality  **dynamic curation strategies**.

# 3. Hyperparameters in Scaling Law

# The Challenge of Hyperparameter Tuning at Scale

123

- Tuning hyperparameters directly on large-scale models?
  - □ Computationally expensive!
- Standard practice in scaling law
  - ○ Tuning on small proxies □ Extrapolating to larger scales
- Problem? **Optimal Hyperparams at small scales may not remain optimal**
  - ○ We need to predict how optimal hyperparameters evolve with model size

# Hyperparameters?

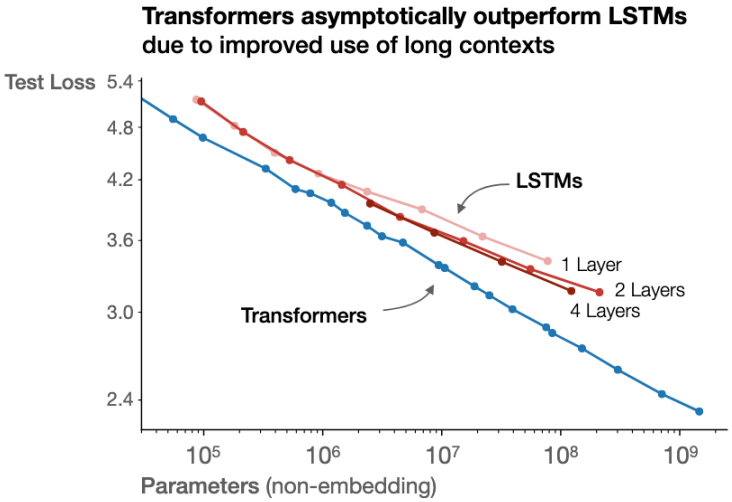
123

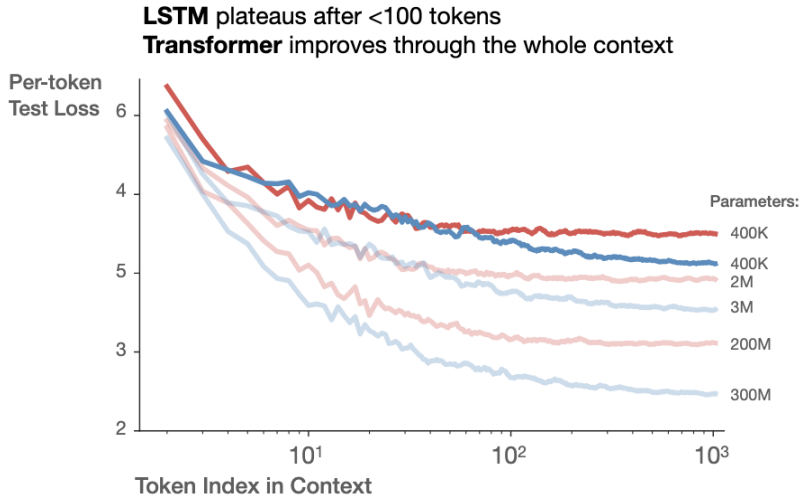
- Architecture
- Optimizer
- Aspect ratio (depth / width)
- Batch size
- Learning rate
- ...

# Architecture: transformers vs LSTMs?



- **Transformers asymptotically outperform LSTMs** as model parameters increase
- Transformers utilize long contexts effectively, whereas LSTMs plateau early

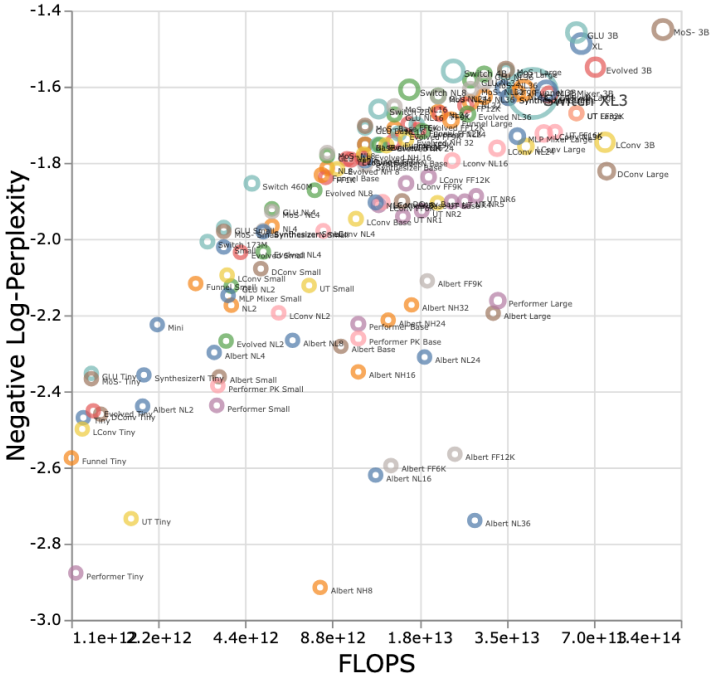


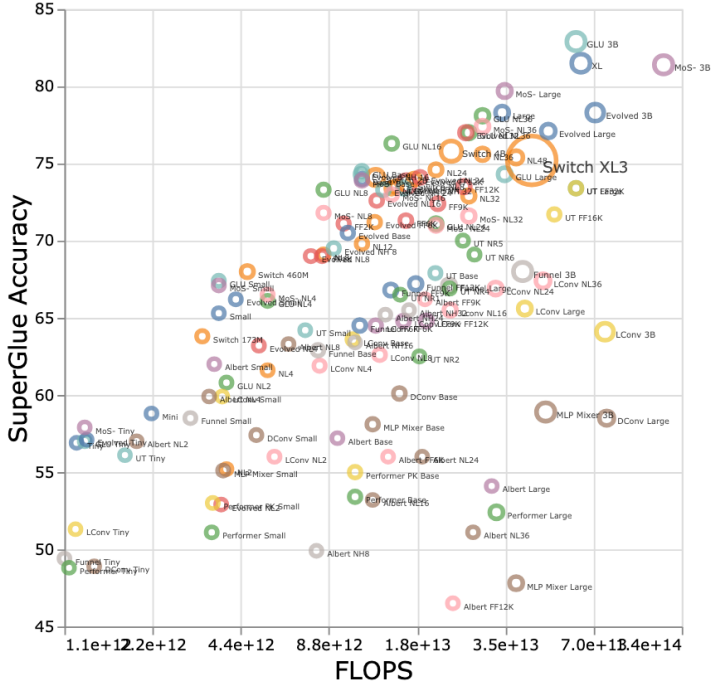

[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# Other Architectures?

123




(a) Upstream: Negative Log-Perplexity



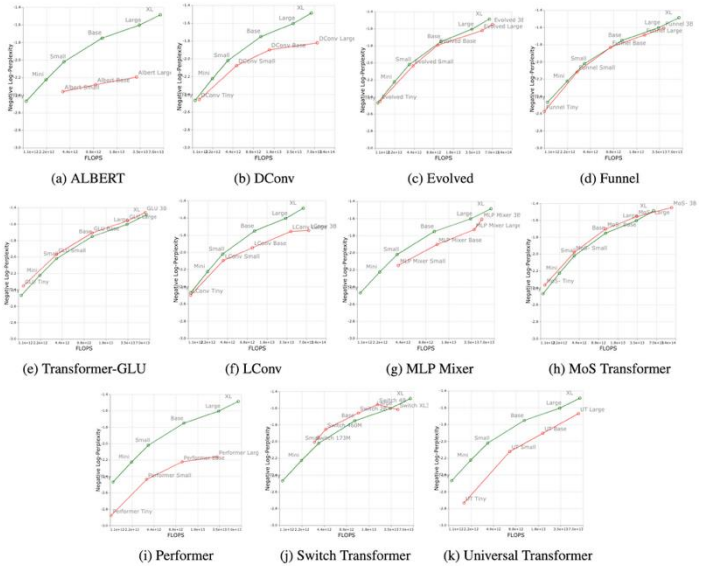

(b) Downstream: Accuracy

[1] Yi, Tay, et al. “Scaling Laws vs Model Architectures: How Does Inductive Bias Influence Scaling?” EMNLP Findings, 2023.

# Other Architectures?

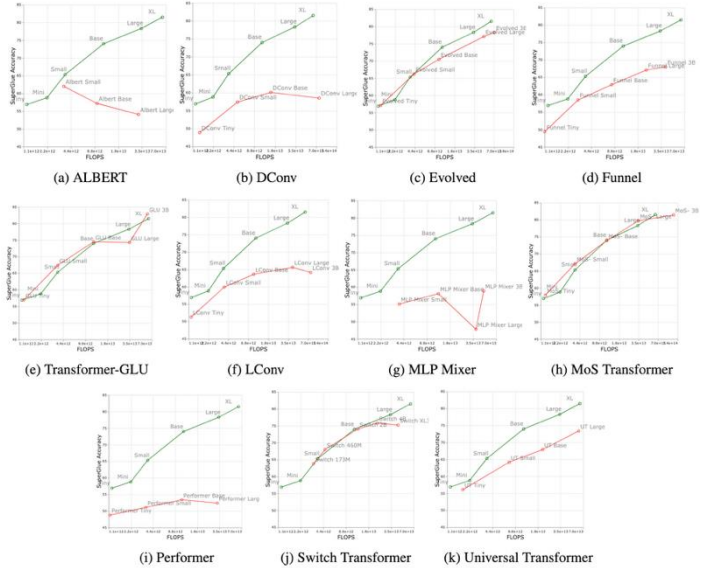
123

- **Vanilla Transformer** vs. **Others**: superior scaling stability and performance




Upstream Negative Log-Perplexity

w

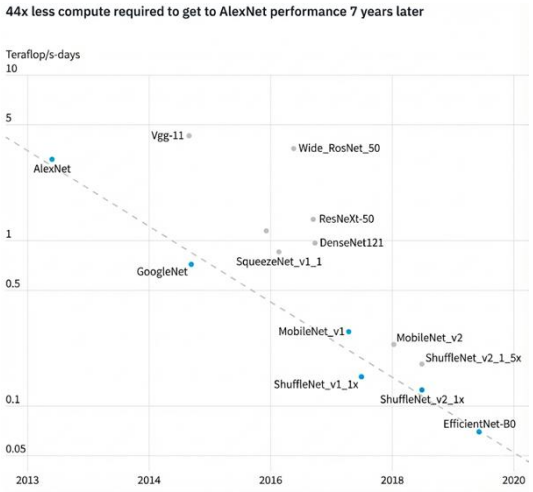



Downstream Accuracy

[1] Yi, Tay, et al. “Scaling Laws vs Model Architectures: How Does Inductive Bias Influence Scaling?” EMNLP Findings, 2023.

# Algorithm innovation in Architecture

- **Algorithmic Efficiency Gains**: Compute required to **reach AlexNet-level accuracy decreased by 44x.**
  - Algorithmic efficiency doubles every 16 months, outpacing the traditional Moore's Law.




[1] Danny, Hernandez, et al. "Measuring the Algorithmic Efficiency of Neural Networks" arXiv, 2020.

# Algorithm innovation in Architecture

123

**Algorithmic innovation is a major driver** alongside hardware scaling

- **Technological Synergies:**
  - Batch Norm, Residual Connections, and Neural Architecture Search, etc.
  - **New techniques synergize** with previous methods to create **exponential efficiency** gains
- Newer architectures consistently dominate older models on compute-performance frontiers

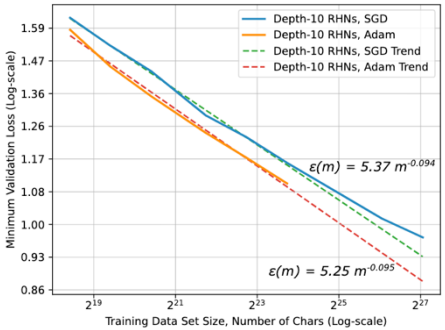
 **Select efficient architectures (Transformer-based) first**, then scale *N* and *D* optimally

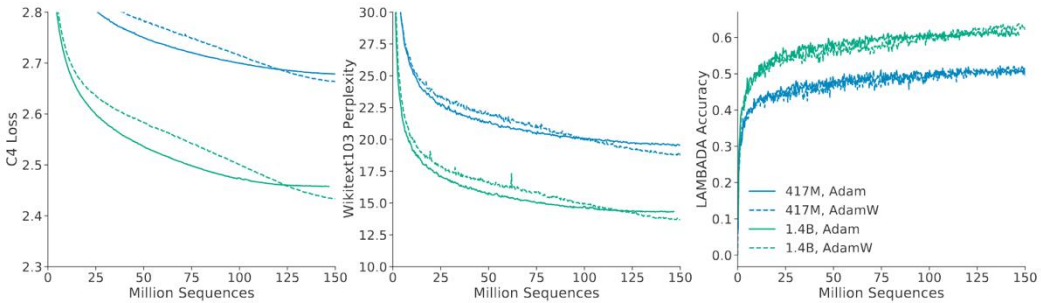
# Optimizer (SGD vs Adam vs AdamW)?

123

- Hestness et al. (2017): **Adam yields better generalization than SGD**
- Chinchilla (2022): **AdamW consistently outperforms Adam** in training loss
  - Independent of the learning rate schedule
  - After 80% of the cosine cycle

- Modern LLMs utilize **AdamW, as it offers superior stability** and downstream performance



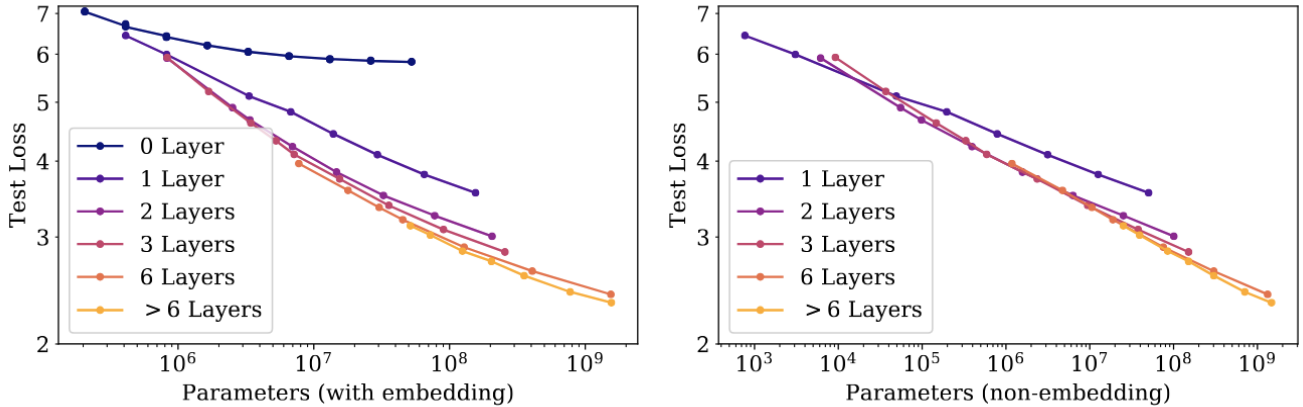


[1] Joel, Hestness, et al. “Deep Learning Scaling is Predictable, Empirically” arXiv, 2017.
[2] Jordan, Hoffmann, et al. “Training Compute-Optimal Large Language Models” NeurIPS, 2022

# Optimal Aspect Ratio (Width vs Depth)?



- Performance depends strongly on N, **weakly on model shape like depth vs width**
- With fixed parameter counts, varying the **aspect ratio has negligible impact on loss**
- **Aspect ratio can vary by a factor of 40** while maintaining similar performance

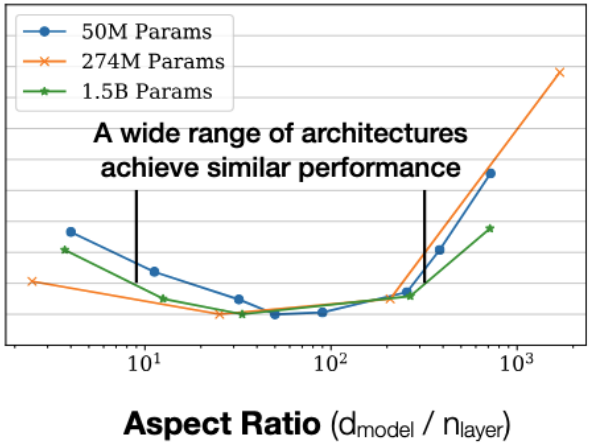



[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# Optimal Aspect Ratio (Width vs Depth)?



- Performance depends strongly on $N$, **weakly on model shape like depth vs width**
- With fixed parameter counts, varying the **aspect ratio has negligible impact on loss**
- **Aspect ratio can vary by a factor of 40** while maintaining similar performance

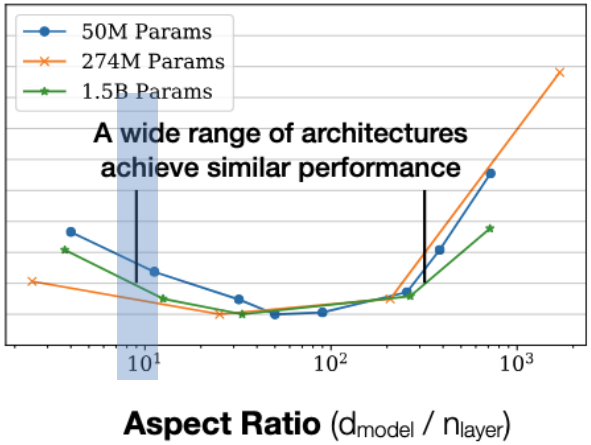



[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# Optimal Aspect Ratio (Width vs Depth)?



- Performance depends strongly on $N$, **weakly on model shape like depth vs width**
- With fixed parameter counts, varying the **aspect ratio has negligible impact on loss**
- **Aspect ratio can vary by a factor of 40** while maintaining similar performance

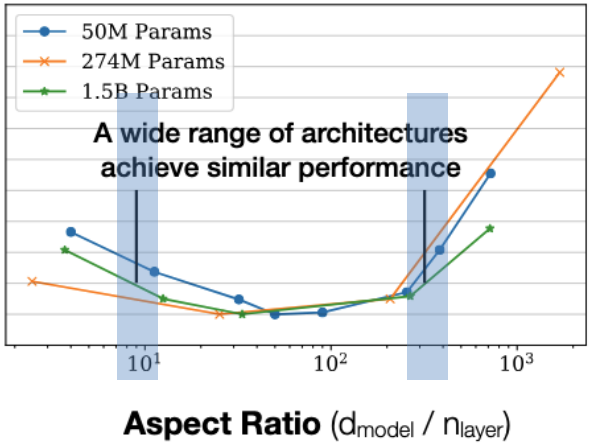



[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# Optimal Aspect Ratio (Width vs Depth)?



- Performance depends strongly on $N$, **weakly on model shape like depth vs width**
- With fixed parameter counts, varying the **aspect ratio has negligible impact on loss**
- **Aspect ratio can vary by a factor of 40** while maintaining similar performance

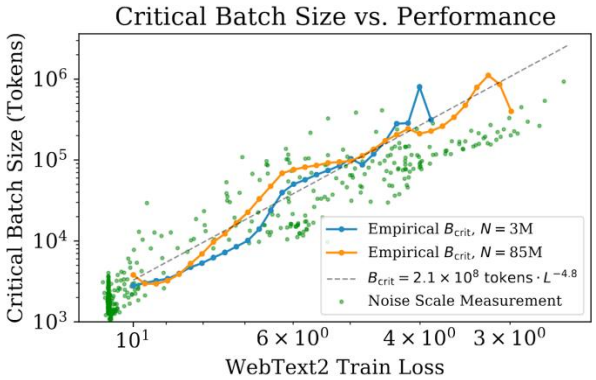



[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# Batch Size?

123

- **Critical batch size optimizes** compute efficiency with training speed via **data parallelism**
- **A power-law relationship with loss**
  - larger batches as the model performance improves
  - depends primarily on the loss rather than model sizes




[1] Jared, Kaplan, et al. “Scaling Laws for Neural Language Models” arXiv, 2020.

# Summary: Hyperparams in Scaling Law

123

- Architecture  **Transformer-based**
- Optimizer  **AdamW**
- Aspect ratio (depth / width)  **Negligible**
- Batch size  **Power-law relationship with loss**
- Learning rate and other hyperparameters  in Lecture 3!
    - Hard to predict
    - Do we still need to tune them on a large scale?

# Questions

123

- Do loss-based scaling laws accurately predict emergent capabilities like reasoning or planning?
- How can we incorporate explicit data quality and diversity metrics into a unified scaling law?
- How scaling laws explicitly predict optimal hyperparameters using minimal small-scale trials?

# What's next?

123

- Advanced Topics in AIs